

Standalone Callable EUR-USD Cross-Currency Swap Pricer

Specification decision: feasible as a transparent Python reference pricer using QuantLib conventions and market objects plus custom vectorized joint simulation and LSM. QuantLib does not supply the complete hybrid engine. “Rigorous” here means an arbitrage-consistent model, controlled numerical error, independent policy evaluation, and documented validation; production use still requires independent model validation and governance.

1. Scope and benchmark trade

The program prices a 10-year, non-call 2-year (10Y NC2) constant-notional cross-currency swap. The benchmark exercise holder receives quarterly compounded EUR €STR and pays semiannual USD fixed 4.00%. USD is collateral, numeraire, and reporting currency; S_t is USD per EUR. USD and EUR notionals are explicit, with initial and final exchanges enabled. Annual zero-fee cancellation decisions occur from year 2 through year 9. On a call date, due cash flows are paid first and only later cancellable flows are removed. All direction, dates, notionals, calendars, lags, fixing conventions, exercise ownership, and event ordering remain explicit JSON fields; reversing pay/receive requires no code change.

Version 1 excludes mark-to-market notionals, stochastic basis, FX smile dynamics, multiple call holders, XVA, collateral switching, partial calls, and historical fixings not supplied in the input. These are different model or product versions, not silent approximations.

2. Files, command, and validation

The executable interface shall be

```
./venv/bin/python standalone_xccy_pricer.py \
--market market_eurUSD.json --deal deal_10y_nc2.json --output result.json
```

Both inputs follow versioned JSON Schemas. The runtime performs dependency-free contract validation before curve construction; the repository also supplies machine-readable schemas for external validation. Unknown fields, inconsistent currency/FX orientation, missing past fixings, non-positive notionals, invalid schedules, and non-positive-semidefinite correlations fail explicitly. Percentages are decimal numbers, dates are ISO-8601, amounts carry currency, and rates/volatilities state their quote convention.

Market input

```
{
  "schema": "xccy-market/1.0", "as_of": "2026-08-14",
  "fx": { "pair": "EURUSD", "quote": "USD_PER_EUR", "spot": 1.10,
    "forwards": [ { "tenor": "2Y", "outright": 1.115 } ] },
  "curves": { "USD-SOFR": { "instruments": [ { "type": "OIS", "tenor": "2Y", "rate": 0.04 } ] },
    "EUR-ESTR": { "instruments": [ { "type": "OIS", "tenor": "2Y", "rate": 0.025 } ] } },
  "swaptions": { "USD": { "type": "NORMAL", "points": [] },
    "EUR": { "type": "NORMAL", "points": [] } },
  "fx_options": { "type": "BLACK_ATM", "points": [] },
  "correlation": { "USD_EUR": 0.25, "USD_FX": -0.15, "EUR_FX": -0.30 }
}
```

Production fixtures contain complete liquid tenors through 10Y; the abbreviated values above define shape, not example calibration data. Bootstrap USD SOFR and EUR €STR curves from OIS quotes and reprice every helper. Under USD collateral, derive the effective EUR discount curve from collateralized EURUSD forwards,

$$P_{EUR}^*(0, T) = P_{USD}(0, T) F_{EURUSD}(0, T) / S_0,$$

and retain the EUR €STR forecast curve through an explicit deterministic forecast/discount basis mapping. Reject inconsistent or under-specified curve/forward data.

Deal input and result

```
{
  "schema": "callable-xccy-deal/1.0", "trade_id": "EURUSD-10Y-NC2",
  "effective_date": "2026-08-18", "maturity": "10Y",
  "reporting_currency": "USD", "collateral_currency": "USD",
  "notionals": { "USD": 100000000, "EUR": 90909090.91 },
  "legs": [
    { "currency": "EUR", "side": "RECEIVE", "index": "ESTR", "frequency": "3M",
      "day_count": "ACT/360", "spread": 0.0, "payment_lag_days": 2 },
    { "currency": "USD", "side": "PAY", "fixed_rate": 0.04, "frequency": "6M",
      "day_count": "30/360", "payment_lag_days": 2 } ],
  "notional_exchanges": { "initial": true, "final": true },
  "exercise": { "style": "CANCEL_REMAINING_SWAP", "holder": "DEAL HOLDER",
    "non_call": "2Y", "frequency": "1Y", "settlement_amount": 0.0,
    "event_order": "PAY_DUE_THEN_CANCEL" }
}
```

The result JSON contains callable and non-callable NPV, option value, standard error and confidence interval, per-leg PV, exercise probabilities, calibration rows, fitted parameters, regression diagnostics, numerical settings, seed, runtime, input hashes, warnings, and software versions.

3. Model under the USD measure

Use one-factor Hull-White states x_d, x_f , deterministic curve-fitting shifts ϕ_d, ϕ_f , and log FX $X = \log S$. With $r_d = \phi_d + x_d$, $r_f = \phi_f + x_f$, one-piece constant rate volatilities, and piecewise-constant FX volatility,

$$dx_d = -a_d x_d dt + \sigma_d(t) dW_d, \quad (1)$$

$$dx_f = [-a_f x_f - \rho_{fS} \sigma_f(t) \sigma_S(t)] dt + \sigma_f(t) dW_f, \quad (2)$$

$$dX = [r_d - r_f - \frac{1}{2} \sigma_S^2(t)] dt + \sigma_S(t) dW_S. \quad (3)$$

The negative foreign quanto drift is for $S = \text{USD}/\text{EUR}$ and the USD money-market numeraire. The shifts fit P_{USD} and P_{EUR}^* ; the foreign curve-fit is defined before transforming its dynamics to $\mathbb{Q}^{\text{USD}}$. The Brownian correlation matrix

$$R = \begin{pmatrix} 1 & \rho_{df} & \rho_{dS} \\ \rho_{df} & 1 & \rho_{fS} \\ \rho_{dS} & \rho_{fS} & 1 \end{pmatrix}$$

must be symmetric positive semidefinite. Correlations are governed inputs and stresses, not residual calibration parameters.

Exact state evolution

On each interval where parameters are constant, evolve the augmented Gaussian state $Z = (x_d, x_f, I_d, I_f, X)$, where $I_c(t) = \int_0^t r_c(u) du$. Use the analytic affine transition $Z_{t+h} = M_h Z_t + m_h + \varepsilon_h$ from the exact OU kernels. Its covariance is the integral of the exact kernel outer products, evaluated by fixed 16-point Gauss-Legendre quadrature. This jointly preserves all rate/FX covariances and produces pathwise discounting $D_d = e^{-I_d}$ without Euler bias. Symmetric eigen factorization is used for the positive-semidefinite covariance; only roundoff-scale negative eigenvalues may be floored.

The event grid contains all accrual boundaries, payments, exchanges, decisions, FX-volatility breakpoints, and settlement dates, with a monthly maximum step. Version 1 represents compounded €STR over each coupon by the continuous-time modeled EUR bank-account accumulation plus the deterministic forecast/effective-curve basis. It therefore supports zero lookback and lockout only; discrete daily-fixing, observation-shift, and lockout conventions require a later product version. Mandatory grid convergence remains an acceptance test.

4. Calibration

Calibration is deterministic for fixed inputs and records market price, model price, error, weight, and optimizer status for every instrument.

1. **Curves.** Bootstrap OIS curves with QuantLib calendars, day counts, settlement conventions, and OIS helpers. Build P_{EUR}^* from FX forwards and test $F = S_0 P_{EUR}^* / P_{USD}$.
2. **Rate models.** For each currency, hold governed mean reversion a_c fixed and fit one positive constant σ_c to the approved co-terminal/diagonal normal-swaption set aligned with the call schedule. QuantLib's Hull-White model and Jamshidian engine price the helpers; calibration minimizes relative price errors and reports every residual. A separately validated model version may add piecewise rate volatility, but version 1 does not infer unsupported buckets.
3. **FX.** Fit positive piecewise $\sigma_S(t)$ to ATM EURUSD Black-volatility tenors. Under the Gaussian-rate/lognormal-FX model, compute each European option from the exact T -forward log-FX variance, including both rate-FX correlations; solve buckets sequentially or by bounded least squares. Re-implied Black vols are reported. A strike smile cannot be fitted by this model and remains diagnostic only.

Default success gates are OIS helper error ≤ 0.01 bp, equal-weight swaption RMS $\sqrt{N^{-1} \sum_i (V_i^{\text{model}} / V_i^{\text{market}} - 1)^2} \leq 5\%$, and FX re-implied-vol error ≤ 0.25 volatility percentage points. A failed gate sets status `CALIBRATION_WARNING` and `accepted=false`; any returned valuation fields are research diagnostics, not an official price.

5. Cash flows and Bermudan LSM

For each path, construct signed USD fixed flows, compounded EUR €STR flows, and notional exchanges without look-ahead; convert EUR payments at contemporaneous path FX and discount with D_d . At decision t_i , split value into common surviving flows A_i , exercise settlement J_i , and cancellable continuation tail Y_i . For this zero-fee cancellation $J_i = 0$. Exercise when

$$J_i > \hat{C}_i + \varepsilon, \quad \hat{C}_i \approx \mathbb{E}^{\mathbb{Q}^{\text{USD}}}[Y_i \mid \mathcal{F}_{t_i}],$$

and add A_i outside the comparison. A call stops only future cancellable flows; it never deletes paid, fixed-surviving, or delayed common flows.

Training pass. On training paths, work backward. The regression target includes discounted cancellable cash flows to the next call plus the already-optimized later value. The version-1 basis contains a constant, standardized $x_d, x_f, \log S$, their squares, and pairwise products. Use QR/SVD least squares with declared ridge fallback. Store scaling, rank, condition number, coefficients, residual RMS, and path count; an ill-conditioned design activates the recorded ridge fallback.

Pricing pass. Freeze the policy and apply it forward to an independently seeded pricing sample. Average all stopped cash-flow streams and calculate standard error from pathwise PVs. The estimate is an LSM lower bound for the holder's option. Retrain across seeds and compare each policy on a common frozen pricing sample. Report unconditional and survivor-conditional exercise rates, no-call probability, and expected exercise date.

6. Numerical controls and acceptance

Vectorized NumPy arrays, antithetics, chunked simulation, and deterministic seeds provide a portable reference implementation. QuantLib remains responsible for dates, conventions, OIS bootstraps, and benchmark marginal pricing; the joint process, cash-flow construction, and LSM are explicit Python because the complete hybrid callable-XCCY engine is not native QuantLib functionality. Minimum automated gates are:

- no call dates reproduce an independent deterministic non-callable XCCY PV within $\max(3SE, 0.25 \text{ bp of USD notional})$;
- zero-volatility and identical-currency limits reproduce deterministic and single-currency results;
- one call date agrees with an independent European or high-accuracy nested-MC benchmark within $\max(3SE, 0.50 \text{ bp})$;
- USD-discounted domestic assets and USD-discounted, FX-converted EUR assets pass martingale tests within three standard errors;
- prices and exercise profiles converge across path count, grid, regression basis, and training seeds; paired comparisons use the standard error of pathwise differences;
- representative multi-call cases bracket the LSM lower bound with a dual upper bound or trusted nested-MC benchmark; and
- holder callable value minus non-callable value equals the reported option value and is non-negative within three comparison standard errors.

The supplied runnable example defaults to 10,000 training and 30,000 pricing paths, antithetics, a monthly maximum model step, and fixed seeds. These are demonstration settings, not production tolerances; final settings are chosen only after convergence, normally with materially larger independent samples. Unit tests cover JSON validation, calendars, FX orientation, exact log-FX variance, curve and calibration repricing, zero-volatility cash flows, martingales, LSM policy freezing, value identities, and reproducibility.

7. Deliverable boundary

The implementation can be completed in Python with no QuantLib source changes. It will be a reviewable reference pricer, not a claim of production approval. Production acceptance additionally requires independent trade replication, market-data governance, model-risk sign-off, performance/soak testing, and reconciliation against a trusted library or trading-system benchmark.

Primary references: Hull and White (1990); Longstaff and Schwartz (2001); QuantLib source and documentation; Open Source Risk Engine/QuantExt cross-asset model as an independent architectural reference.